

RISE4impact

Change report: what was fixed, who did it, and how each item was verified
Covering 15 August to 5 October 2026 · generated 5 October 2026

Why this document exists

This records every change made to RISE over this period, separated into work done in the codebase and work only Neil could do in an account. Every claim below was re-tested against the live site immediately before this was written, and anything that could not be verified is marked as such rather than assumed.

Two things in here are not fixes. One is an outage caused by an attacker. The other is a database outage caused by a migration in this very list. Both are documented in full, because a change report that only lists successes is not much use the next time something breaks.

Status at a glance

Area	State	How that was established
Site	Healthy	HTTP 200 on apex and www, 0.31s, serving the expected bundle hash
Malware	Removed	Rogue Worker and its route deleted; zero injection markers in the served HTML
Sign-in	Working	Broke twice, fixed twice; a real signup was accepted and a profile row created
Spend cap	Real	Shared in KV across isolates; 19 of 70 concurrent requests refused by the edge limiter
Admin gate	Holding	17/17 tests; unknown action 400, unauthenticated call refused, with the service key present
Backups	None	Not completed. The only item on the list that cannot be fixed after the fact
Breach check	Unavailable	Requires the Supabase Pro plan; password length and character rules are on

The two red rows are the honest state of things, not oversights. Both are explained in the open items section at the end.

Part 1 · The two outages

Neither was a routine bug, and the second was self-inflicted. They come first because they are the most useful things in this document.

1.1 Someone put malware in front of the site

On 19 September, visitors to `rise4impact.org` were shown a fake reCAPTCHA telling them to paste a command into their terminal. The command downloaded and executed a remote script.

The repository was never touched. The local `index.html` was clean at 7,785 bytes while the edge served 10,156. The extra 2,371 bytes were appended after the closing body tag by a *second Cloudflare Worker*, named `worker-billowing-tooth-7c4f`, routed at `*rise4impact.org/*` in front of the real one. It fetched a base64 payload from a BNB Smart Chain contract and appended it to every HTML response.

The payload lived on a blockchain rather than a server, so its author could change what it did at any time without touching the site again. The technique is known as EtherHiding.

Why every check said the site was clean

The injected script began by detecting whether it was being watched:

```
isHeadless() || isLocalhost()  
? console.log("stop watching us :)")  
: isWindows ? load_("0x46790e2Ac7...")  
: isMac ? load_("0x68DcE15C10...")
```

Every scan run against the site was headless or curl, so the script stayed dormant and showed nothing. The evidence that broke the case was a screenshot from Neil's own browser. **A real user's report was worth more than every automated check.**

What was done about it

Action	Detail
Site taken offline	A maintenance page served from Worker code rather than assets, which also proved the injection happened after the Worker ran
Rogue Worker deleted	Route <code>ec7ba8d6</code> and script <code>worker-billowing-tooth-7c4f</code> removed through the Cloudflare API
Content-Security-Policy added	<code>script-src</code> allows this origin plus one inline block named by its sha256. The injected script carried no hash and would have been refused by the browser whatever appended it
connect-src restricted	Only this origin and Supabase. A script that somehow ran still could not reach a blockchain RPC to fetch its payload
Cloudflare tokens deleted	Three build tokens, each with 20+ permissions across all zones. The likely way in

The lesson worth keeping: the edge is part of the attack surface. Nothing in a code review would have found this, because the code was never modified.

1.2 A migration in this list broke sign-in

Migration 005 set an empty search path on all eight database functions. Before writing it, every function was checked for unqualified *table* names, and all were already qualified. **Type names were not checked.** Five functions cast to custom enums by bare name:

```
handle_new_user 'volunteer'::account_role
submit_opportunity cur opportunity_status
withdraw_opportunity cur opportunity_status
submit_application v_role account_role
withdraw_application v_cur application_status
```

An empty search path cannot resolve a type any more than it can resolve a table. Each function raised an error the moment it ran. `handle_new_user` fails inside the signup trigger, which Supabase reports as `unexpected_failure`, giving no hint that a search path was the cause.

The blast radius was wider than sign-in. Posting an opportunity, withdrawing one, applying, and withdrawing an application were all broken the same way. Nobody could create an account on RISE by any method for as long as this was live.

Migration 006 repaired it by setting those five to `pg_catalog, public, pg_temp`, which resolves the types while still naming `pg_temp` last. That matters: `pg_temp` is searched first for relation and type names when it is not listed, and any signed-in user can create objects there, so the original `= public` left a shadowing route open. The three functions that touch nothing but built-ins keep the empty path.

This was found because Neil tried to sign in, not because anything flagged it. It was also described as safe to run on a live database before it was run.

Part 2 · Fixes made in the codebase

All committed and pushed to github.com/koderking25/rise-netlify1, and all live.

2.1 Security

Fix	What was wrong	Verified by
Spend cap made real	DAILY_BUDGET_USD and the rate limit were module-scope variables, meaning one copy per Cloudflare isolate, each starting at zero. Five dollars a day was in fact five dollars per isolate per day	KV total accumulating across calls; 19 of 70 concurrent requests refused
Rate limit moved to the edge	40 requests a minute was 40 per isolate. Traffic from several countries got a fresh allowance in each	Burst test, refusals counted
handle_new_user RPC closed	PostgREST publishes every public function as a REST endpoint. The function that assigns account roles, running as SECURITY DEFINER, was callable by anyone unauthenticated	Revoked in 005 and in 001 for fresh installs
Search paths pinned	Five functions set search_path = public, which still searches pg_temp first; three set nothing at all	All eight confirmed set; see 1.2 for the correction
CSP and security headers	No CSP at all, which is how the injected script was able to run	Headers confirmed live on the apex
CORS locked to the two origins	ALLOW_ORIGIN unset, so the header fell back to *, letting any website spend the Anthropic budget through its visitors	Unknown origin receives the canonical origin, not its own

A bug in the obvious fix, worth recording. The review being worked from advised setting ALLOW_ORIGIN to both spellings comma-separated. That header accepts exactly one origin, and a list is rejected by every browser, so following the advice literally would have stopped RISE calling its own API. The request's Origin is now matched against the list and echoed back.

2.2 The matching engine

Rebuilt so that results carry a link to the actual application form rather than an organization's homepage. Every change below came from running it against the live API and watching it fail, five times.

Failure observed	Cause	Fix
Timed out at 125s with a 524, full price, nothing returned	A deep search held open as one non-streaming request	Every call streams and is reassembled server-side
Spent 11 turns reporting it had hit a limit and would continue next turn	Exhausting the search budget left it stuck	The prompt states there is no next turn
Returned 10 organizations with no links at all	The fix for the above overcorrected: told to stop when a tool refused, it stopped fetching too	Search and fetch are separate budgets; running out of one is the cue to spend the other
Omitted every application link	Told only to report what it had read, it declined to offer pages it had actually opened	Acceptable answers ranked: the form, then the portal, then the volunteer page. Never a homepage
Wrote 22,889 tokens of commentary, then was cut off before emitting	Narration consumed the output budget	Silence between tool calls is the rule; max_tokens raised from 8,192 to 32,000
Zero results reported as an honest empty search	A truncated emit arrives as {} which is truthy, so an expensive failure looked like a successful empty result	Empty results are now treated as failure

Measured result, Brantford: 10 roles, 8 with a real application page, all 8 verified reachable, 2 dead links dropped automatically. Real URLs such as brantcountyspca.com/volunteer-form/ and the hospital's own application page, with age limits and screening read off those pages rather than guessed. The words likely, typically and usually are banned in those fields, because a guessed age floor turns a 14 year old away from a door that would have taken them.

Cost, and how it is held down

A deep run costs about 60 cents. Judging moved to Opus 5 with thinking; discovery stays on Sonnet. That split is deliberate: discovery reads the web and is by far the largest input in the app, while judging sees a short candidate list and is pure judgement. The expensive model runs where the tokens are few and the thinking changes the answer.

The cohort cache is the real lever. Discovery is impersonal, keyed on place and category, and held for 24 hours. **One paid run serves every student in that town for the day.** Verified as a miss then a hit on the same key, byte-identical, and a miss on a different one.

Pricing was also charging Opus calls at Sonnet rates, billing the spend guard roughly 40% of their real cost, which is the wrong direction for a budget to be wrong in.

2.3 Reliability and speed

Fix	Before	After
Refresh no longer signs people out	Any failure destroyed the stored session. A network blip, a 5xx, or simply having RISE open in two tabs logged you out of both	Single-flight refresh; cleared only on an explicit rejection
Domain binding in the repo	The hostname existed only as dashboard state, so deleting it was silent and total. The site went completely dark on 19 September	Declared in wrangler.toml; a deploy rebuilds the DNS record
App split from the HTML	98% of a 732KB page was the inline app, re-downloaded on every visit	8KB shell plus a hashed bundle cached for a year
React self-hosted	Loaded from a third-party CDN, which school networks often block, hanging on Loading RISE forever	Served from this origin
Dead links dropped	Results could point at pages that 404	Checked before a student ever clicks
Database kept awake	Supabase pauses Free projects after seven days of low activity, silently	A daily cron at 12:00 UTC

2.4 Product and documentation

Change	Detail
Email drafter removed	As requested
Disability communities named separately	Blind and low vision, Deaf and hard of hearing, and people with disabilities are now distinct options. CNIB and Canadian Hearing Services are not interchangeable, and the roles differ
Hidden layer added	Typing rise, piano or ember does something, as does the Konami code. Never fires while you are typing, and respects prefers-reduced-motion
README corrected	It opened by saying index.html was the whole app, which stopped being true and was the sentence most likely to send someone editing the wrong file
SECURITY.md written	Records the five Advisor findings that are the design working correctly, so nobody breaks the view that is the only thing standing between volunteers and the opportunities table
GO-LIVE.md written	The step-by-step guide for the account work

Part 3 · What Neil did

These needed an account only Neil could sign into, or a password only he had.

Task	Status	Verified by
Restored the site after the outage	Done	wrangler login, then deploy from the project directory. Correctly stopped when wrangler offered to publish his entire home folder
Installed the Supabase service key	Done	Admin endpoint returns the five verification bar items
Topped up the Anthropic account	Done	Live calls succeed and report cost
Ran migrations 004, 005 and 006	Done	Signup accepted and a profile row created after 006
Set password length and character rules	Done	Short passwords and single-class passwords refused; see below
Deleted the over-permissioned Cloudflare tokens	Done	Account now lists one Worker and no stray routes
Anthropic monthly spend limit	Not done	The only control that does not depend on this repository being correct
Database backup	Not done	Blocked by a password mix-up; see open items

A save that silently failed

The Supabase auth settings page let a Pro-only toggle be switched on, then rejected the entire form when saved. Password length and character rules went down with it, so three settings appeared set and none were. This was only caught by attempting real signups and reading which ones were refused.

```
abc12 -> rejected: at least 8 characters
aB3xyzQ -> rejected: at least 8 characters
alllowercase99 -> rejected: missing a character class
ALLUPPERCASE99 -> rejected: missing a character class
NoDigitsHere0k -> rejected: missing a character class
```

Both rules are confirmed live. The gap that remains: Password123 satisfies every rule available on the Free plan and is near the top of every breach list. Only the HaveIBeenPwned check catches that, and it needs Pro.

Part 4 · Verification, run 5 October

Every line below was executed against production immediately before this document was generated.

Check	Result
Apex and www	200, 0.31s, 8,416 bytes
Bundle served matches the repo	app.f4d3059369.js on both
Injection markers in served HTML	0
Content-Security-Policy	present, script-src pinned to one sha256
nosniff, DENY, HSTS, referrer-policy	all present
Spend and rate guards	sharedSpend: true, edgeRateLimit: true
Admin: get-bar	200
Admin: unknown action	400, allowlist holding
Admin: no token	Sign in required.
CORS, unknown origin	not echoed back
Password rules	length and character class both enforced
Email confirmation	required
Worker secrets	all four bound
Cloudflare scripts and routes	one script, no stray routes
Admin gate tests	17/17 passed
Git	0 uncommitted, 0 unpushed

Not verifiable from here

Three test accounts created during password testing, at risetest+NNNNN@mailinator.com, may still exist. They cannot be queried without the service key, which lives only as a Worker secret. Removing them:

```
delete from auth.users where email like 'risetest+%@mailinator.com';
```

Part 5 · Open items

Item	Why it matters	Effort
Take a database backup	There are none, and the Free plan does not provide them. Today the database was broken by a migration and recovered by luck: the repair happened to be five lines of SQL. A backup is what makes the next one survivable. The only item that cannot be fixed after the fact.	15 min
Anthropic monthly spend limit	Auto-reload is pointed at a card. Every other guard depends on this repository being correct; this one does not	2 min
Two-factor on all four accounts	Supabase, Cloudflare, GitHub, Google. How the attacker got in was never established	15 min
Delete the three test accounts	One was created with a breached password	1 min
Free disk space	3.4 GB remaining. Already blocked one approach to the backup	varies
Consider the Pro plan	Daily backups, no auto-pausing, and the breach check. Three problems, one decision, \$25 a month	decision

Two structural notes

The application is one 16,000-line file. Readable and well organised, but with no module structure, so every change touches the same file. This is the main thing that will slow RISE down as it grows.

Test coverage is thin where it matters most. `submit_application`, `submit_opportunity` and `enforce_named_verifier` are the three places where a mistake has a safeguarding consequence rather than a cosmetic one, and none of them has a test. Today's broken migration would have been caught instantly by one.

Where the code is

What	Path
Repository	/Users/neilmekouar/rise claude code
Remote	github.com/koderking25/rise-netlify1 (branch main)
The app	public/app.f4d3059369.js · 578 KB
Page shell	public/index.html · 8 KB
Worker entry	src/index.js · CSP, routes, keep-alive cron
AI proxy	functions/api/ai-search.js
Admin endpoint	functions/api/admin.js
Migrations	supabase/001 through 006, run in order
Build step	node build.mjs after editing the bundle

Run `node build.mjs` **after editing the bundle.** Three things must agree: the bundle's filename hash, the reference to it in `index.html`, and the CSP script hash in `src/index.js`. Edit the bundle without it and the site serves a file nobody points at, or a page whose inline script the browser refuses.